SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Yellampati Swathi	**Reg. No.:**	192324095
Section / Batch:		**Date:**	27-10-2026

Code of Conduct

I Yellampati Swathi (192324095) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Y. Swathi

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Yellampati Swathi
192324095

Section 1: Resume Information Extraction

Aim: Extract candidate details, identify technical skills, calculate eligibility, and generate a structured profile summary.

Program:

Python Program

```
import re

resume = """
Name: Rahul Kumar
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""

name = re.search(r"Name:\s*(.*)", resume)
email = re.findall(r"[\w.-]+@[\w.-]+\.\w+", resume)
mobile = re.findall(r"\b[6-9]\d{9}\b", resume)
experience = re.search(r"Experience:\s*(\d+)\s*years?", resume)

skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
found_skills = []

for skill in skills:
    if re.search(re.escape(skill), resume, re.I):
        found_skills.append(skill)

years = int(experience.group(1)) if experience else 0
candidate_name = name.group(1).strip() if name else "Not found"

print("Candidate Name:", candidate_name)
print("Email:", email)
print("Mobile:", mobile)
print("Technical Skills:", found_skills)
print("Experience:", years, "years")
```

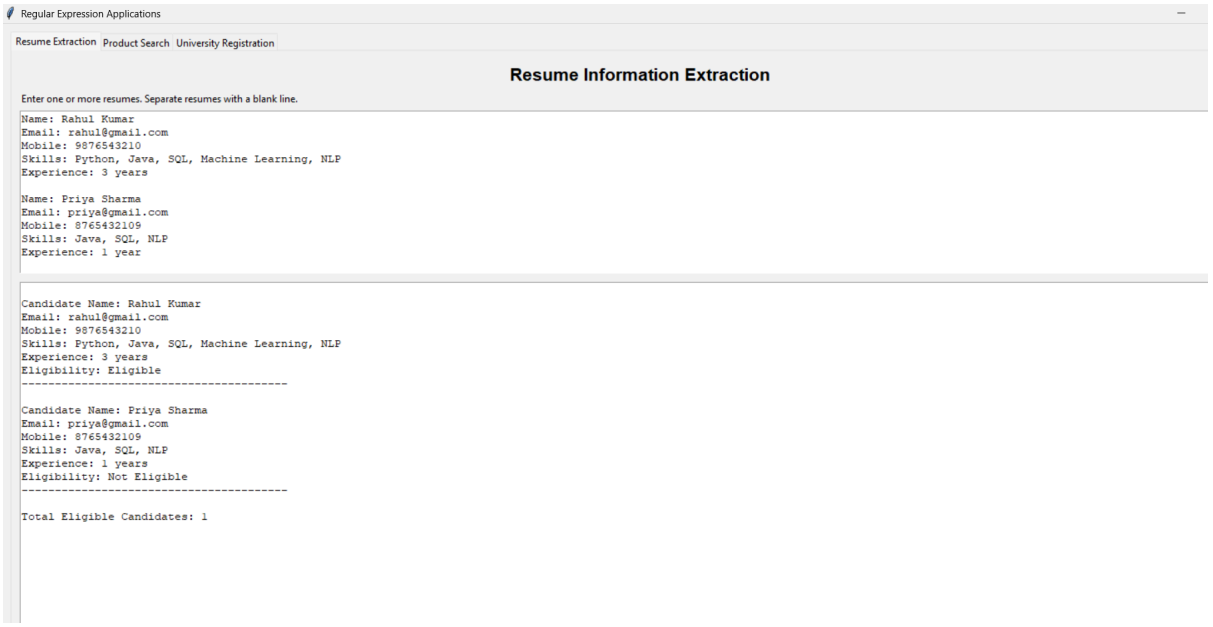
```
print("\nCandidate Profile Summary")
print("Name:", candidate_name)
print("Skills:", ", ".join(found_skills))
print("Experience:", years, "years")

if years >= 2 and "Python" in found_skills:
    print("Eligibility: Eligible")
else:
    print("Eligibility: Not Eligible")
```

Sample Output

```
Candidate Name: Rahul Kumar
Email: ['rahul@gmail.com']
Mobile: ['9876543210']
Technical Skills: ['Python', 'Java', 'SQL',
                   'Machine Learning', 'NLP']
Experience: 3 years

Candidate Profile Summary
Name: Rahul Kumar
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
Eligibility: Eligible
```



Result: The program extracts resume information and identifies candidates with at least 2 years of experience and Python skills.

Section 2: Product Search System

Aim: Search products using exact, prefix, suffix, partial, and case-insensitive matching and display the number of matching products.

Program:

Python Program

```
import re

products = [
    "Laptop",
    "Laptop Bag",
    "Smartphone",
    "Smart Watch",
    "Wireless Mouse",
    "Wireless Keyboard",
    "Laptop Stand"
]

keyword = input("Enter search keyword: ")

exact = []
prefix = []
suffix = []
partial = []

for product in products:
    if re.fullmatch(re.escape(keyword), product, re.I):
        exact.append(product)

    if re.match(re.escape(keyword), product, re.I):
        prefix.append(product)

    if re.search(re.escape(keyword) + r"$", product, re.I):
        suffix.append(product)

    if re.search(re.escape(keyword), product, re.I):
        partial.append(product)

print("\nExact Matches:", exact)
print("Prefix Matches:", prefix)
print("Suffix Matches:", suffix)
print("Partial Matches:", partial)
```

```
print("\nSearch Report")
print("Exact Matches:", len(exact))
print("Prefix Matches:", len(prefix))
print("Suffix Matches:", len(suffix))
print("Partial Matches:", len(partial))
```

Sample Output

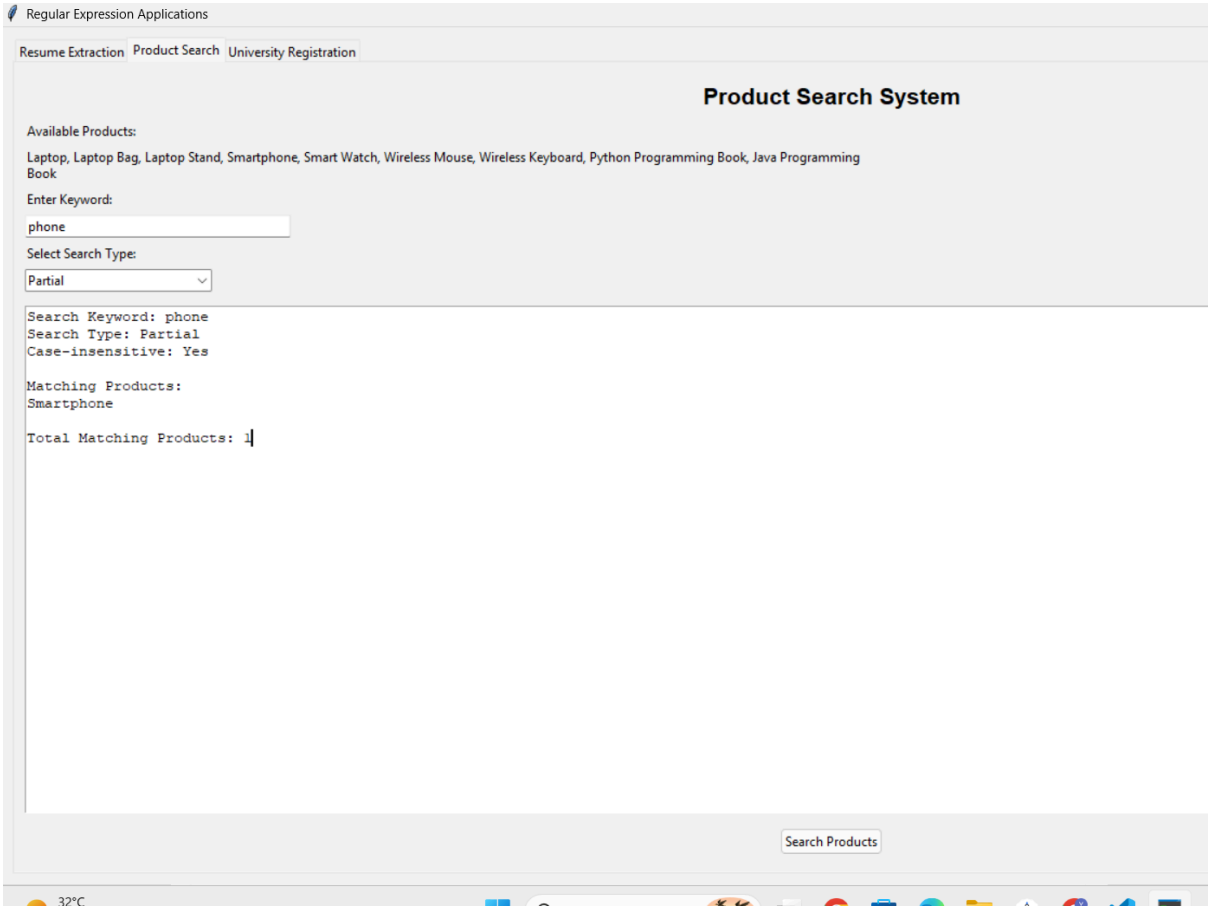
Input:

Enter search keyword: Laptop

Output:

Exact Matches: ['Laptop']
Prefix Matches: ['Laptop', 'Laptop Bag', 'Laptop Stand']
Suffix Matches: ['Laptop']
Partial Matches: ['Laptop', 'Laptop Bag', 'Laptop Stand']

Search Report
Exact Matches: 1
Prefix Matches: 3
Suffix Matches: 1
Partial Matches: 3



Result: The program searches products using different matching methods and generates a report of matching product counts.

Section 3: University Registration System

Aim: Validate the student register number, institutional email, course code, semester, and mobile number using Regular Expressions.

Program:

Python Program

```
import re

reg_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

r1 = re.fullmatch(r"\d{2}[A-Z]{3}\d{4}", reg_no)
r2 = re.fullmatch(r"[\w.-]+@university\.edu", email)
r3 = re.fullmatch(r"[A-Z]{3}\d{3}", course)
r4 = re.fullmatch(r"[1-8]", semester)
r5 = re.fullmatch(r"[6-9]\d{9}", mobile)

print("\nValidation Report")

print("Register Number:",
      "Valid" if r1 else "Invalid")
print("Institutional Email:",
      "Valid" if r2 else "Invalid")
print("Course Code:",
      "Valid" if r3 else "Invalid")
print("Semester:",
      "Valid" if r4 else "Invalid")
print("Mobile Number:",
      "Valid" if r5 else "Invalid")

if r1 and r2 and r3 and r4 and r5:
    print("\nRegistration Status: Successful")
else:
    print("\nRegistration Status: Failed")
```

Sample Output

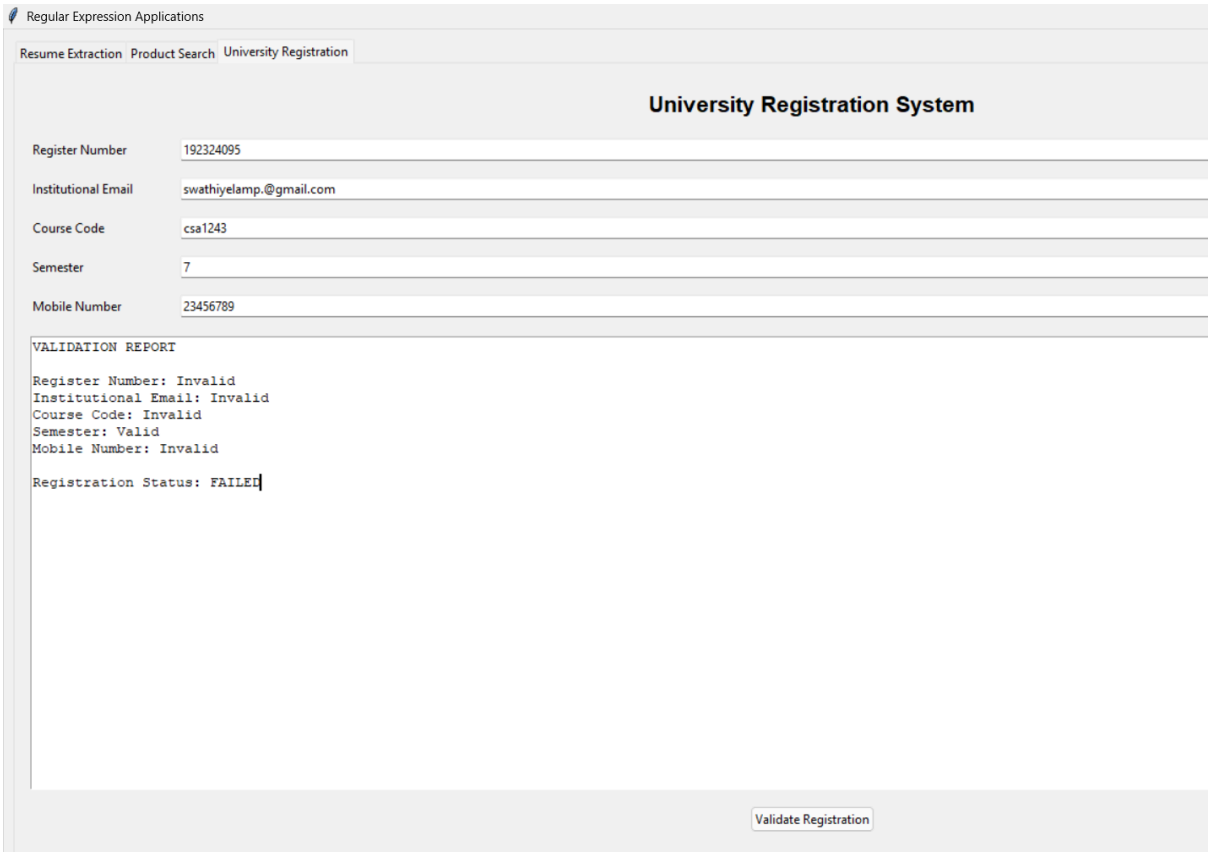
Input:

Enter Register Number: 22CSE1234
Enter Institutional Email: rahul@university.edu
Enter Course Code: CSE101
Enter Semester: 5
Enter Mobile Number: 9876543210

Output:

Validation Report
Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

Registration Status: Successful



Result: The program validates all student registration fields and displays whether registration is successful.